

Financial Statement Extraction Engine

Product Requirements Document

Prepared for **Antigravity**
Date 24 May 2026
Version 1.0 · Confidential

1. Document Information

Project name	Financial Statement Extraction Engine
Client	Antigravity
Version	1.0
Status	Ready for Development
Date	24 May 2026
Author	Product Team
Classification	Confidential

2. Executive Summary

This document defines the requirements for a scalable, automated pipeline that extracts structured financial data from company PDF annual reports. The system must handle PDFs across three distinct types — digital (text-embedded), scanned, and photograph-based — and produce a consistent, validated JSON output per company per fiscal period, regardless of each company's proprietary formatting.

The extracted output targets four canonical financial fields: Revenue, Cost of Materials Consumed, Change in Inventory, and EBITDA. The system is designed to process 1,000+ companies with multiple PDFs per year at minimal per-document cost, with a strong constraint that all extracted values must be traceable to source pages and arithmetically verified.

3. Problem Statement

3.1 Current State

Financial analysts currently extract income statement data from annual reports manually. Each company publishes its report in a proprietary format. There is no standard layout, no consistent row-label vocabulary, and no common unit convention. Reports arrive as digital PDFs, scanned documents, and photographs of physical pages. Manual extraction at scale is slow, expensive, and error-prone.

3.2 Key Challenges

- PDF type heterogeneity: the same company may submit a digital PDF one year and a scanned photograph the next
- Label variance: 'Revenue from operations', 'Net turnover', 'Gross billings', and 'Primary activity income' all refer to the same canonical field
- Format variance: tables appear in different positions, with different column structures, across 1,000+ companies
- Noise: 500-page PDFs contain auditor reports, MD&A; prose, notes to accounts — only ~5–15% of pages contain relevant financial tables

- Hallucination risk: any LLM-generated value that is not grounded in the actual extracted document is a data integrity failure

4. Objectives

4.1 Primary Objectives

- Extract the four canonical financial fields from any company PDF with >90% field-level accuracy
- Handle digital, scanned, and photograph PDF types in a single unified pipeline
- Produce a structured JSON output with value, unit, confidence score, and source page per field
- Scale to 1,000 companies × 5 PDFs/year × 500 pages without manual intervention on >90% of documents
- Ensure zero hallucinated values — every extracted number must be traceable to a source page

4.2 Secondary Objectives

- Keep per-PDF processing cost below \$0.50
- Provide a human review queue for low-confidence extractions
- Build a self-improving alias dictionary that reduces LLM dependency over time
- Return results queryable across all companies and periods via a structured database

5. Scope

5.1 In Scope

- PDF ingestion from Azure Blob Storage
- Automated classification of PDF type (digital / scanned / photograph)
- Table extraction using appropriate method per type
- Canonical field mapping for the four target financial fields
- Arithmetic cross-validation of extracted values
- Confidence scoring and conflict detection
- Human review queue UI for low-confidence results
- Structured JSON output storage in PostgreSQL
- REST API to query extracted data by company and fiscal period

5.2 Out of Scope

- Balance sheet or cash flow statement extraction (Phase 2)
- Real-time PDF processing (batch processing is sufficient for v1)
- Multi-language document support (English-only for v1)
- Direct integration with broker or data vendor APIs

6. Canonical Output Schema

Every successfully processed PDF produces one JSON record per fiscal period detected. The schema is fixed regardless of the source company or document format.

```
{
  "company_id": "ACME_LTD",
  "fiscal_period": "FY2024",
  "source_pdf": "acme_annual_report_2024.pdf",
  "extracted_at": "2024-11-01T09:32:00Z",
  "revenue": {
    "value": 12500000,
    "unit": "INR_lakhs",
    "confidence": "high",
    "source_pages": [42, 87],
    "alias_matched": "revenue from operations"
  },
  "cost_of_materials_consumed": { ... },
  "change_in_inventory": { ... },
  "ebitda": { ... },
  "arithmetic_check_passed": true,
  "conflicts": [],
  "unresolved_fields": []
}
```

Confidence levels are defined as:

- **high** — value consistent across multiple pages, arithmetic check passed
- **medium** — single source page, arithmetic check passed
- **low** — single source, arithmetic check failed or value inferred — routed to human review

7. Pipeline Architecture

The pipeline is divided into five sequential stages. Stages 1–2 are type-specific. Stages 3–5 are shared across all PDF types.

Stage 1 — PDF Classification

Tool: PyMuPDF

Detect whether the PDF contains embedded text (digital), raster images (scanned), or embedded photograph images. Route to the appropriate extraction branch. This stage adds no cost and takes under 1 second per PDF.

Stage 2A — Digital Extraction

Tool: pdfplumber

Extract tables and text from text-embedded PDFs. Returns rows, columns, and raw cell values. Free, deterministic, handles ~50% of the expected volume. No API calls required.

Stage 2B — Scanned Extraction

Tool: pdf2image + Gemini Flash Vision

Convert each page to an image and send to Gemini 2.0 Flash Vision for table extraction. Returns structured JSON of rows and values. Cost ~\$0.00008 per page.

Stage 2C — Photograph Extraction

Tool: OpenCV + Gemini Flash Vision

Apply perspective correction, shadow normalisation, and adaptive binarisation before sending to Gemini Flash Vision. If the preprocessing quality check fails (insufficient page boundary detection), the page is routed directly to human review.

Stage 3 — Arithmetic Traversal + Label Matching

Tool: Python + rapidfuzz + sentence-transformers

Identify the canonical field for each extracted row using: (1) alias dictionary fuzzy match, (2) embedding similarity against canonical labels, (3) positional heuristics based on arithmetic chain structure. No API cost. Handles ~75–80% of rows.

Stage 4 — Exception Mapping

Tool: Gemini Flash (text)

Rows that did not resolve in Stage 3 are sent to Gemini Flash with the full table context. The model maps ambiguous labels to canonical fields. Prompt explicitly prohibits inference or calculation — model maps only, returns null if uncertain. Applies to ~10–15% of rows.

Stage 5 — Validation + Output

Tool: Python + PostgreSQL

Arithmetic cross-check: $\text{Revenue} - \text{COGS} \approx \text{Gross Profit}$, $\text{Gross Profit} - \text{OpEx} \approx \text{EBITDA}$. Values that fail the check are flagged. Confidence scores are assigned. Final JSON is written to PostgreSQL. Low-confidence records are added to the human review queue.

8. Anti-Hallucination Design

Preventing fabricated values is a primary design constraint, not a secondary concern. The following mechanisms are layered to ensure every value in the output is traceable to the source document:

8.1 LLM sees extracted values, never raw PDF

The LLM at Stage 4 receives only the table JSON produced by Azure/pdfplumber/Gemini Vision — a structured list of real cell values already extracted from the document. It cannot fabricate a number because it has no access to the PDF itself; it only performs label mapping on data already in front of it.

8.2 Explicit null instruction

The Stage 4 prompt contains a hard constraint: "Return null for any field not explicitly present in this table. Do not infer, estimate, interpolate, or calculate any value." The model is evaluated on this constraint during testing.

8.3 Arithmetic cross-validation

Every output is cross-checked: Revenue minus all deduction rows must equal the stated EBITDA within a defined tolerance. If it does not, the record is flagged as low-confidence and queued for human review — it is never silently accepted.

8.4 Source page traceability

Every field in the output carries a `source_pages` array. A downstream user or auditor can open the PDF to the stated page and verify the value directly. Any value without a source page cannot enter the output schema.

8.5 Deterministic stages do the heavy lifting

Stages 1, 3, and 5 contain zero LLM calls. The LLM is invoked only for genuine label ambiguity (~10% of rows). The more a value is handled by deterministic code, the lower the hallucination risk.

9. Technology Stack

Layer	Technology	Purpose	Cost model
PDF storage	Azure Blob Storage	Stores all incoming PDFs	~\$0.02/GB/month
Job queue	Azure Service Bus	Triggers processing per PDF	~\$0.05/M messages
Workers	Azure Container Apps	Autoscaling processing pool	Pay-per-use, scales to zero
Digital extraction	pdfplumber (Python)	Table extraction from text PDFs	Free / open source
Image conversion	pdf2image + poppler	PDF page to image for OCR branch	Free / open source
Photo preprocessing	OpenCV (Python)	Deskew, shadow removal, binarise	Free / open source
OCR / Vision	Gemini 2.0 Flash Vision	Table extraction from images	\$0.10/M tokens
Exception mapping	Gemini 2.0 Flash	Ambiguous label resolution	\$0.10/M input tokens
Embeddings	sentence-transformers (local)	Semantic label similarity	Free, runs on CPU
Fuzzy match	rapidfuzz (Python)	Alias dictionary matching	Free / open source
Database	PostgreSQL	Canonical output store	~\$50/month managed

Layer	Technology	Purpose	Cost model
API	FastAPI (Python)	Query interface for output data	Runs on Container Apps
Review UI	React + FastAPI	Human review queue interface	Runs on Container Apps

10. Cost Model

Assumptions: 1,000 companies, 5 PDFs per company per year, average 500 pages per PDF. PDF type split: 50% digital, 30% scanned, 20% photograph. LLM exception rate: 10% of all pages.

Component	Applies to	Volume	Unit cost	Annual cost
pdfplumber extraction	Digital PDFs (50%)	1.25M pages	\$0	\$0
Gemini Flash Vision (scanned)	Scanned PDFs (30%)	750K pages	\$0.00008/page	~\$60
OpenCV + Gemini (photo)	Photo PDFs (20%)	500K pages	\$0.0001/page	~\$50
Gemini Flash text (exceptions)	~10% of all pages	250K pages	\$0.0001/page	~\$25
Azure Blob Storage	All PDFs	~2 TB	\$0.02/GB/month	~\$480
Azure Container Apps	Worker pool	Dynamic	Per vCPU-second	~\$200
PostgreSQL (managed)	Output store	Persistent	Flat	~\$600
Azure Service Bus	Job queue	5K messages/yr	~\$0.05/M	~\$1

Metric	Value
Total annual cost (1,000 companies, 5K PDFs)	~\$1,400 / year
Cost per PDF	~\$0.28
Cost per company per year	~\$1.40
LLM cost as % of total	~6%
Infrastructure cost as % of total	~91%

11. Functional Requirements

ID	Requirement	Priority	Notes
FR-01	System shall classify each incoming PDF as digital, scanned, or photograph within 5 seconds	Must	PyMuPDF-based, no API call
FR-02	System shall extract all table structures from digital PDFs using pdfplumber	Must	No API cost
FR-03	System shall convert scanned PDF pages to images and extract tables via Gemini Flash Vision	Must	
FR-04	System shall apply OpenCV preprocessing (deskew, shadow removal, binarisation) to photograph pages before OCR	Must	
FR-05	System shall run a preprocessing quality check on photograph pages and route failures to human review	Must	
FR-06	System shall match extracted row labels to canonical fields using alias dictionary and fuzzy matching first	Must	rapidfuzz, threshold 85%
FR-07	System shall use sentence-transformer embeddings as a secondary matching layer	Must	Local model, no API
FR-08	System shall use Gemini Flash for rows unresolved after FR-06 and FR-07	Must	~10% of rows only
FR-09	Gemini Flash prompt must explicitly prohibit inference, estimation, or calculation	Must	Prompt-level constraint
FR-10	System shall perform arithmetic cross-validation on every extracted record	Must	Tolerance: $\pm 2\%$
FR-11	Every output field must carry value, unit, confidence, and source_pages	Must	
FR-12	Records with confidence=low must be added to human review queue	Must	
FR-13	Human reviewer confirmations must update the alias dictionary	Must	Feedback loop
FR-14	System shall expose a REST API to query outputs by company ID and fiscal period	Must	FastAPI
FR-15	System shall support parallel processing of multiple PDFs simultaneously	Should	Azure Container Apps scaling
FR-16	System shall retain all intermediate extraction artefacts for 90 days for audit	Should	Blob storage

12. Non-Functional Requirements

Category	Requirement
Performance	A 500-page digital PDF must complete end-to-end in under 3 minutes
Performance	A 500-page scanned PDF must complete end-to-end in under 8 minutes
Throughput	System must support 50 concurrent PDF processing jobs
Accuracy	Field-level extraction accuracy must exceed 90% on a benchmark set of 50 labelled PDFs
Traceability	100% of output values must carry a source_page reference
Reliability	System must gracefully handle corrupt or password-protected PDFs without crashing
Cost	Per-PDF processing cost must remain below \$0.50
Scalability	Worker pool must scale to zero when idle and to 50 workers under load
Auditability	All LLM calls must be logged with input, output, model version, and timestamp
Security	No PDF content may be stored in LLM provider logs beyond the API call window

13. Recommended Build Sequence

Build in phases to validate each layer before adding complexity. Do not build the photograph branch until the digital branch is proven in production.

Phase	Duration	Scope	Exit criterion
Phase 1	Weeks 1–2	Digital PDFs only: pdfplumber + arithmetic traversal + alias matching	80%+ field accuracy on 20 real digital PDFs
Phase 2	Weeks 3–4	Add scanned PDF branch: pdf2image + Gemini Flash Vision	Pipeline handles scanned PDFs with same output schema
Phase 3	Weeks 5–8	Add photograph branch: OpenCV preprocessing + quality gate + Gemini Flash Vision	Preprocessing passes quality check on 80%+ of photo PDFs
Phase 4	Weeks 9–10	Human review queue UI + alias dictionary feedback loop	Reviewer corrections persist and reduce future LLM calls
Phase 5	Weeks 11–12	REST API, PostgreSQL query layer, monitoring, cost dashboards	API live, costs tracked per company and per PDF type

14. Human Review Queue

The human review queue is not a failure mode — it is a designed component of the pipeline. It handles three categories of records:

- Photograph pages where OpenCV preprocessing failed the quality check
- Records where arithmetic cross-validation failed and could not be auto-resolved
- Any field with confidence=low after all automated stages

14.1 Reviewer workflow

The review UI presents the reviewer with the source PDF page image alongside the extracted table and the proposed field mapping. The reviewer can accept, correct, or reject each field. Accepted or corrected mappings are written back to the output and the alias dictionary is updated with the confirmed label → canonical field mapping.

14.2 Self-improvement loop

Every human correction teaches the system. After processing 200 companies, the alias dictionary will contain the majority of label variants seen in the real corpus. The LLM exception rate should drop from ~10% to ~3–4% of rows as the dictionary matures.

15. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Photograph quality too poor to preprocess	Medium	High	Quality gate routes to human review immediately; do not attempt extraction on failed images
Company uses non-standard EBITDA definition (pre/post exceptional items)	High	Medium	Flag in conflicts[], include all candidate values, require human confirmation on first encounter
Multi-segment statement — consolidated vs standalone on same page	Medium	High	Prefer 'Consolidated' labelled table; if ambiguous, extract both and flag
Gemini API rate limits under high concurrency	Low	Medium	Implement exponential backoff; queue overflow to secondary batch window
Alias dictionary grows stale as companies change label names	Low	Low	Re-run alias matching on each new PDF; dictionary is append-only
PDF encryption or DRM prevents extraction	Low	High	Detect at classification stage, route to human review with notification

16. Open Questions for Antigravity

- What is the expected SLA for PDF processing — same-day batch or near-real-time?
- Are PDFs fetched from a known source (BSE/NSE filing portals) or uploaded manually by users?
- Should the human review queue be an internal admin tool or accessible to client analysts?

- Is there a preferred cloud region for Azure infrastructure (data residency)?
- What is the benchmark set — can 50 labelled PDFs with correct values be provided for accuracy testing?
- Should the system handle PDFs in languages other than English in a future phase?
- Is there an existing company master (company_id, name, exchange code) to join against, or does the system create it?

17. Acceptance Criteria

The system is considered production-ready when all of the following are true:

- Field-level extraction accuracy $\geq 90\%$ on a benchmark set of 50 labelled PDFs spanning digital, scanned, and photograph types
- 100% of output values carry a non-null source_pages reference
- Arithmetic cross-validation passes on $\geq 85\%$ of records without human intervention
- Per-PDF cost does not exceed \$0.50 averaged over a 100-PDF test batch
- A 500-page digital PDF completes end-to-end in under 3 minutes
- Human review queue correctly receives and stores reviewer corrections, and alias dictionary updates are reflected in subsequent runs
- REST API returns correct structured JSON for a given company_id and fiscal_period query

This document is confidential and prepared solely for Antigravity. All cost estimates are indicative based on current public pricing as of May 2026 and subject to revision.